

abdul4rehman215

GenAI Detection-as-Code CI/CD for Wazuh - Full MVP Prototype Overview

A multi-workflow SOC automation prototype integrating GitHub, n8n, Wazuh, OWASP LLM, ATLAS, Slack, TheHive, audit DataTables, dashboards, and closure sync.

Project done by: [Abdul Rehman \(abdul4rehman215\)](#)

GitHub: [abdul4rehman215](#)

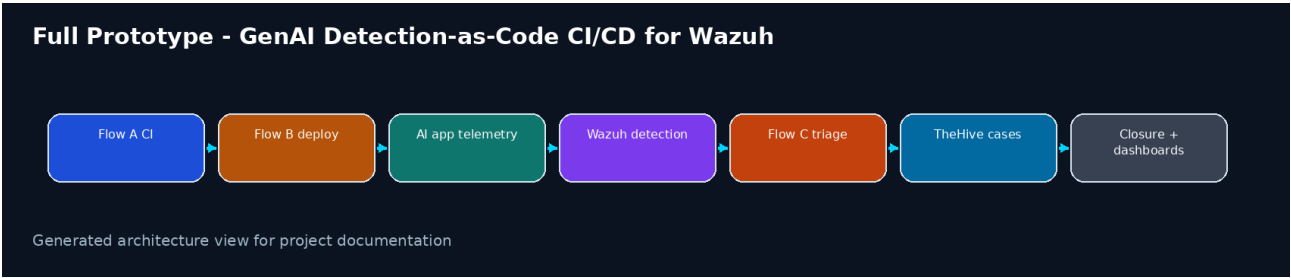
Document Type Master prototype document Full project overview	Prototype Focus GenAI security operations Detection-as-code Wazuh CI/CD TheHive case management
Project Character Portfolio-grade MVP prototype with implementation and validation evidence	Design Goal Showcase a complete detection lifecycle from PR validation to runtime triage and case-closure feedback

Why this document matters

- This write-up works both as a portfolio project document and as an implementation-facing prototype note.
- It explains the problem, the workflow design, the SOC value, and the validation evidence that proves the workflow worked in the lab.
- The screenshots are real build/test artifacts collected from n8n, GitHub, Wazuh, Slack, TheHive, and n8n DataTables.

Key outcomes

- Combines Flow A, Flow B, Flow C, and supporting workflows into one connected GenAI SOC automation operating model.
- Demonstrates detection engineering, controlled deployment, runtime AI security triage, case management, and audit dashboards in a single prototype.
- Provides evidence from GitHub, n8n, Wazuh, Slack, TheHive, DataTables, AI demo app, and the dashboard collector.



Master architecture view - the complete MVP lifecycle from detection content PR to runtime detection, case management, and closure sync.

1. Executive Summary

This capstone MVP demonstrates a realistic detection-as-code and runtime GenAI SOC automation pipeline. It was built as a portfolio prototype, but the architecture reflects production-style security operations: code review and CI, gated deployment,

runtime detections, analyst triage, case management, audit tables, dashboard metrics, error handling, and closure synchronization.

2. Problem Statement

- GenAI application risks are emerging quickly, but SOC workflows often do not have a clean path from AI telemetry to detection engineering, deployment, triage, and case closure.
- Detection rules can break if deployed without validation; runtime alerts can be noisy if not enriched; case outcomes can drift if not synchronized back to the operations layer.
- A modern SOC prototype should show the full lifecycle: build detection content, validate it, deploy it safely, detect runtime abuse, triage it, create a case when necessary, and preserve closure state.

3. Complete Workflow Architecture

Flow A - Detection CI	Validates PR detection content and publishes CI report, labels, Slack, and DataTable/dashboard evidence.
Flow B - Controlled Deployment	Deploys approved Wazuh rules/decoders through a gated backup/stage/check/restart/postdeploy/rollback lifecycle.
Flow C - Runtime GenAI Triage	Converts AI demo app guardrail events into Wazuh detections, Slack triage, TheHive alerts/cases/comments, and audit rows.
Supporting Workflows	Collect dashboard events, capture errors/dead letters, and synchronize TheHive case closure back into Flow C state.
Shared evidence layer	n8n DataTables and Slack notifications make the system demonstrable, auditable, and easy to review.

4. Workflow Evidence Overview



Figure O1 - Flow A full workflow canvas: PR CI validation and audit path.



Figure O2 - Flow B full workflow canvas: gated deployment and rollback-aware reporting.



Figure O3 - Flow C full workflow canvas: runtime alert intake, TheHive/Slack/audit path.

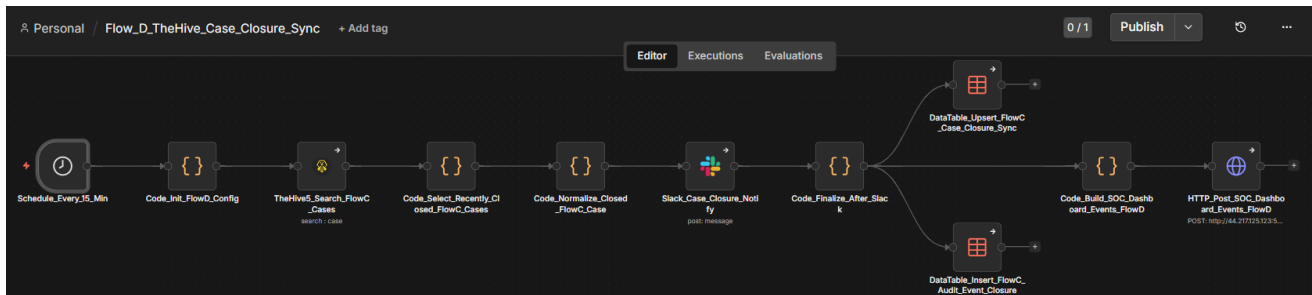


Figure O4 - Supporting workflow evidence: case closure sync / dashboard / error support layer.

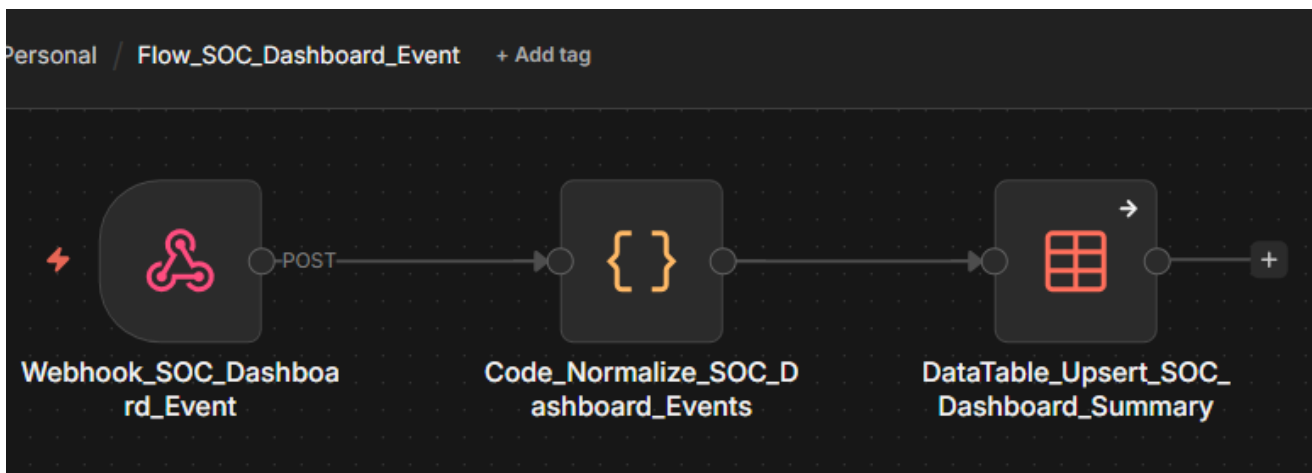


Figure O5 - SOC dashboard collector workflow canvas.



Figure O6 - Global error/dead-letter workflow canvas.

```

root@n8n:/opt/detection-ci/wazuh-genai-ci# ls -lh
total 36K
-rw-r--r--  1 ubuntu ubuntu   16 Apr 22 18:01 README.md
drwxr-xr-x  3 root    root    4.0K Apr 24 21:04 app
drwxr-xr-x  4 ubuntu ubuntu   4.0K Apr 22 10:10 detections
drwxr-xr-x  3 root    root    4.0K Apr 25 20:13 docs
drwxr-xr-x  2 root    root    4.0K Apr 24 21:08 mappings
drwxr-xr-x  2 ubuntu ubuntu   4.0K Apr 24 21:07 metadata
drwxr-xr-x  2 ubuntu ubuntu   4.0K Apr 24 21:08 schemas
drwxr-xr-x  4 ubuntu ubuntu   4.0K Apr 26 12:08 scripts
drwxr-xr-x  4 ubuntu ubuntu   4.0K Apr 25 20:13 tests

```

Figure 07 - Repository/scripts overview demonstrating the underlying implementation assets.

5. End-to-End Operating Model

- A pull request introduces or updates detection content.
- Flow A validates the content and labels the PR with detection-ci-pass only when safe.
- Flow B deploys only when the PR is approved, ready-to-deploy, CI-passed, and triggered through a valid deployment signal.
- Flow C detects runtime GenAI abuse through Wazuh and turns it into analyst-ready Slack and TheHive context.
- Supporting workflows preserve dashboard metrics, error evidence, and TheHive case closure state.

6. Tools and Why They Were Used

GitHub	Source of truth for detection content, PRs, comments, labels, and approval/deploy gates.
n8n	SOAR/orchestration engine connecting GitHub, shell scripts, Wazuh, Slack, TheHive, and DataTables.
Wazuh	Detection engine for AI application telemetry and target GenAI rules.
AI demo app	Controlled runtime source that emits guardrail telemetry for testing direct/indirect injection and unsafe output cases.
Slack	Analyst notification and operational visibility channel.
TheHive 5	Alert and case management platform for triage, templates, comments, and closure state.
n8n DataTables	Structured audit and status layer for CI runs, deployments, alerts, cases, closures, dashboard metrics, and errors.

7. SOC Value: MTTD, MTTR, and Analyst Productivity

- MTTD improves because AI app telemetry is immediately ingested by Wazuh and pushed into n8n triage.
- MTTR improves because high-risk detections are enriched, linked to TheHive, and promoted to cases with templates and comments automatically.
- Alert fatigue is reduced by filtering specific GenAI rule IDs and carrying the recommended action, risk flags, user/session/request context, and TheHive links into Slack.
- Response consistency improves because deployment gates, case templates, audit rows, and dashboard metrics enforce a repeatable workflow.

8. Key Differentiators / USPs

- End-to-end lifecycle: code validation -> controlled deployment -> runtime detection -> triage -> case promotion -> closure sync.
- Not just a Wazuh demo: includes CI/CD, SOAR, TheHive, dashboards, and evidence-backed audit tables.
- GenAI-specific runtime detections mapped to OWASP LLM and ATLAS language.
- Portfolio-ready prototype with real screenshots and validation outputs from every layer.

9. Prototype Maturity and Limitations

- This is an MVP prototype designed for a lab environment, not a hardened enterprise production deployment.
- The AI app intentionally simulates common GenAI abuse patterns instead of integrating a production LLM gateway.
- The dashboard is currently a structured metric table; a full visual BI dashboard can be added later.
- Security hardening such as secret rotation, private network restrictions, and multi-environment promotion would be required before production use.

10. Future Enhancements

- Add more OWASP LLM categories such as sensitive information disclosure, insecure plugin/tool use, and model denial-of-service.
- Add visual dashboards over the SOC summary table and deployment/audit tables.
- Add branch protection/status checks from Flow A directly into GitHub required checks.
- Add TheHive webhook-based closure sync instead of scheduled polling when available.
- Add environment promotion such as dev -> staging -> production Wazuh managers.

11. Conclusion

This MVP demonstrates a full GenAI detection engineering and SOC automation lifecycle. It shows practical builder-level capability across detection-as-code, Wazuh operations, n8n orchestration, Slack notification, TheHive case management, DataTable audit design, and operational dashboarding.